Phase 6 分析笔记：正确性、复杂度、数值边界、盈亏平衡

日期：2026-10-11 · **对应**：handout Phase 6 · **代码**：`scripts/screening.py`、`scripts/benchmark.py`

数据：MVTec AD bottle/tile/screw，16-shot（库 12,544 patch = 16 图 × 784 patch × 1536 维）

1. 正确性

1.1 命题

设 M 为参考库，$c_j \in M$ 为 cell 中心，$C_j = \{m \in M : c_j = \operatorname{argmin}_{c} d(m,c)\}$ 为 cell，$r_j \geq \max_{m \in C_j} d(m,c_j)$。对任意 q：

$$L(q) = \min_j \max\{0, d(q,c_j) - r_j\} \;\leq\; s_M(q) = \min_{m\in M} d(q,m) \;\leq\; U(q) = \min_j d(q,c_j)$$

1.2 证明（上界）

对任意 j，$d(q,c_j) \geq s_M(q)$（因为 $c_j \in M$，而 s_M 是到 M 的最小距离）。

所以 $U(q) = \min_j d(q,c_j) \geq s_M(q)$。∎

1.3 证明（下界）

固定任意 $m \in M$，设它属于 cell j。由三角不等式 $d(q,m) \geq d(q,c_j) - d(c_j,m) \geq d(q,c_j) - r_j$。

又 $d(q,m) \geq 0$，故 $d(q,m) \geq \max\{0, d(q,c_j) - r_j\}$。

由于该式对每个 m 成立，$s_M(q) = \min_m d(q,m) \geq \min_m \max\{0, d(q,c_{j(m)}) - r_j\} \geq \min_j \max\{0, d(q,c_j) - r_j\} = L(q)$。∎

（最后一步：m 跑遍 M 时 j(m) 跑遍所有非空 cell，所以 min over m ≥ min over j。）

1.4 决策不变性

若 $U \leq t$ 则 $s \leq U \leq t$ → 真值不报警，屏幕判"正常"正确。

若 $L > t$ 则 $s \geq L > t$ → 真值报警，屏幕判"报警"正确。

只要 $L \leq s \leq U$ 成立，本地裁决与整库裁决必然一致——这是零错判的唯一依据，不依赖任何统计假设。

1.5 两个反例级别的实现陷阱（本次实测踩到）

陷阱	后果	为什么
用 k-means **均值**当子中心	$L \leq s \leq U$ 失效	均值 $\notin M$，$d(q,c)$ 可能小于 s_M，U 不再是上界
细化时不保留父中心	U 变差（5.696 → 5.890）	父中心被移出候选集，$\min_j d(q,c_j)$ 上升

修法：子中心取 **medoid**，且强制包含父中心。修后实测区间 5.696 → **5.270**（更紧）。

2. 复杂度

操作	复杂度	实测（bottle, N=12,544, D=1536）
贪心核心集	O(N·K·D)	k=32 约 1.3 s，k=1254 约 1.7 s（GPU；首次数值含 CUDA 初始化）
cell 分配 + 半径	O(N·K·D)	含在上行
单 patch 区间	O(K·D)	k=32: 3.41 ms/图（784 patch）
整库单 patch	O(N·D)	9.02 ms/图（GPU）
精确树查询	最坏 O(N·D)，1536 维退化	**8963 ms/图**（见 §5）

操作	复杂度	实测（bottle, N=12,544, D=1536）

构建成本可离线承担（秒级），查询成本随 K 线性增长。

3. 数值边界

- 计算用 **FP32**。实测 GPU float32 与 CPU float64 的最大相对差 **2.38e-07**（W1 数值契约）。
- 半径加保护量 `GUARD = 1e-6`：`r_j ← r_j + GUARD`，使 U 略松、L 略紧，**宁可升级也不误判**。
- 并列：`s == t` 判**正常**（严格大于才报警）。单元测试 3 覆盖。
- 空 cell：中心必在 M 中，正常不会出现；若出现，半径置 0 并保留该中心（不崩溃，单测 4 覆盖）。
- 确定性：GPU 与 numpy 两条核心集路径**逐位一致**（单测 6b），两次构建逐位一致（单测 7）。

4. 字节与盈亏平衡

4.1 屏幕端字节（不含编码器）

K	中心	半径	元数据	合计	占全库
32	196,608 B	128 B	~60 B	**192.2 KB**	0.26%
128	786,432 B	512 B	~60 B	**768.5 KB**	1.02%
1254	7,704,000 B	5,016 B	~60 B	**7,528.9 KB**	10.00%

4.2 时延（逐图，只算检索，不含编码器）

类别	编码器	全库暴力	BallTree	筛查 k=32	k=128	k=1254
bottle	31.8 ms	9.02 ms	**8963 ms**	3.41 ms	4.51 ms	23.70 ms
tile	42.1 ms	8.38 ms	**17352 ms**	3.55 ms	4.48 ms	23.67 ms
screw	34.6 ms	11.61 ms	**15350 ms**	3.38 ms	4.42 ms	24.00 ms

*编码器时延含读盘与解码，是**上界**；筛查实现为 CPU numpy，若上 GPU 会显著更快，因此 k=1254 的 23.7 ms 是**保守**数字。*

4.3 盈亏平衡（远端解析器口径）

筛查总代价 `T_screen + p·(T_transfer + T_resolver)`；整库路径 `T_transfer + T_resolver`。

筛查更快当且仅当

```
p < p* = (T_transfer + T_resolver − T_screen) / (T_transfer + T_resolver)
```

类别	K	p*(T=0)	p*(20 ms)	p*(50 ms)	p*(100 ms)	实测升级率	是否划算
bottle	32	0.622	0.882	0.942	0.969	98.8%	✗
bottle	1254	0.000	0.183	0.599	0.783	10.8%	✓（T≥20 ms）
tile	32	0.577	0.875	0.939	0.967	98.3%	✗
tile	1254	0.000	0.166	0.595	0.782	61.5%	✗（刚好超过）
screw	32	0.709	0.893	0.945	0.970	100%	✗
screw	1254	0.000	0.241	0.610	0.785	15.6%	✓（T≥20 ms）

结论：代价结构是"粒度 vs 升级率"的权衡。

- 粗粒度（k=32）省内存到 0.26%，但几乎每张都升级 → 不划算。
- 细粒度（k=1254，7.5 MB）在**远端解析器 + 传输 20 ms 以上**时对 bottle/screw 划算，tile 刚好不划算。
- 本地解析器（T=0）下 k=1254 永不划算，因为它比 GPU 暴力还慢（实现原因，非原理原因）。

5. 一个重要的负面发现：精确树在高维退化

`sklearn.BallTree`（精确欧氏最近邻）在 1536 维下：

类别	建树	查询（折算每图）	与暴力最大差
bottle	2.4 s	**8963 ms**	9.6e-05
tile	2.4 s	**17352 ms**	4.5e-06
screw	2.4 s	**15350 ms**	3.5e-05

比 GPU 暴力慢约 1000 倍，而距离值完全一致（差异来自 float32）。

含义：计划书里"用精确树做等资源对照"这一步不能当作"更省的替代品"来写，

树在 1536 维不是免费的午餐；真正的对照应该是"更小的库"或"更小的编码器"。

6. 尚未解决的问题

问题	影响	处理
Phase 3 的升级率（bottle 8.4%）与本模块（10.8%）差 2.4 pp	同一指标两个数	**以本模块为准**，重跑 Phase 3 筛查列
筛查实现为 CPU numpy	k=1254 时延被高估	论文中标注为保守值；若要更准，移植到 GPU
传输时延为受控假设	无真实边缘链路	报告中标注为仿真，给出 0/20/50/100 ms 四点
仅 16-shot	升级率依赖库覆盖	Phase 7 补 1/4/40-shot 与多种子

7. 复现

```
python scripts\screening.py --selftest
python scripts\benchmark.py --category bottle --k 32 128 1254
python scripts\benchmark.py --category tile   --k 32 128 1254 --out logs\w6_benchmark_tile.json
python scripts\benchmark.py --category screw  --k 32 128 1254 --out logs\w6_benchmark_screw.json
```